

PURPOSE: Reinforce the concepts of iterative vs recursive algorithms and their complexities and actual timings.

PROBLEM: For this program, you will randomly generate integers for a dynamically allocated array. You will make a copy of the array and perform the iterative merge sort on the first copy and the recursive merge sort on the second copy. You will then search for a key in the *now sorted* first array using iterative binary search and then again with binary search. You will time how long it takes to execute the algorithms in nanoseconds. Specific requirements:

- Use the filenames specified in this assignment. Copy code in file that is available in D2L.
- Declare and implement the NINE functions listed below. Figure out what parameters you need.

```
void openFile();
void printPageHeading();
void printColumnHeadings();
void iBinSearch(); //use the new improved iterative binary search
void rBinSearch(); //use the recursive binary search from CMPS 1063
void iMergeSort(int * x, long left, long right);
void rMergeSort(int * x, long left, long right);
void Merge(int * x, long left, long middle, long right);
void genValues();
```

- Open output file and print page heading
- Print column headings
- Put these values, 100, 1000, 10000, 100000, 1000000 in an array or vector.
- Using a for loop, execute the following steps $i = 5$ times
 - Dynamically allocate array based on size $[i]$ (100, 1000, 10000, 100000, 1000000).
 - Randomly generate values to put in the array.
 - Dynamically allocate another array the same size and copy its values.
 - Compute $\log_2 n$ and $n \log_2 n$
 - Run and individually **time** each of the 2 merge sort algorithms
 - Run and individually time each of the 2 binary search algorithms (for the searches, use $x = 13$ as your search key).
 - Print a line to the output for this size (n is $\text{size}[i]$)
 - Deallocate the dynamic arrays of values
- Figure out averages over the 5 trials either by

NOTES:

- Use long for indices in the dynamically allocated array, and long long or long double for the timings.
- Use for loop(s).
- Values must be neatly formatted - double and long double can be printed out with `setprecision(0)`.
- Use my code for `iMergeSort`, `rMergeSort`, and `Merge` (but study them for Exam 3 and the Final). I will create a .cpp file with these in them and you can start with that file.
- Look at my sample code for doing timings and for generating random numbers.
- Don't forget to deallocate dynamically allocated arrays.
- Create a table similar to the sample output below. NOTE: $n \log_2 n$ is a long long.

INPUT FILE: No input file, your program will randomly generate the numbers to put in an array.

SAMPLE OUTPUT: *Stringfellow_Prog4_1.out* for *Stringfellow_Prog4.cpp*:

Catherine Stringfellow
Program 4: Algorithm Timing Comparisons

n	log2 n	iBin	rBin	nlog2n	iMerge	rMerge
100	6	300	300	600	3200	63300
1000	9	300	400	9000	6200	629600
10000	13	700	800	130000	167200	6731000
100000	16	800	1000	1600000	611400	72881500
1000000	19	1900	1200	19000000	6555900	719640500

TO DO TIMINGS

```
#include <chrono>    //for high resolution clock
#include <time.h>     //clock, clock_t declared here

IN MAIN
//Use the high_resolution_clock to time in nanoseconds
using Clock = chrono::high_resolution_clock;

//variables for timings
clock_t start, stop;

//time code

//get start
start = Clock::now().time_since_epoch().count();

//call function here for time algorithm

//get stop and compute elapsed time
stop = Clock::now().time_since_epoch().count();
elapsedTime = (double)(stop - start);
```

TO RANDOMLY GENERATE DATA

```
#include <stdlib.h> //srand, rand

IN MAIN GENVALUES

//initialize random seed
srand(time(NULL));

//generate a random value
value = rand();
```

TURN IN (UPLOAD TO D2L):

- *Lastname_Prog4.cpp*
- FIVE Output files: *lastname_Rec1.out* to *lastname_Rec5.out* from *Lastname_Prog4.cpp*
OR you can write a big loop that executes the body of main 5 times and turn in one output. You put the loop *after* you open the output file and print the report heading and end it before you close the file.
- **Create a report** (either produced by your program or done in a table in Word or Excel). This report should sum your timings for the different algorithms and array sizes and average them. You should **explain the results** that you get. Below is what my averages look liked.

Average Timings

n	log2 n	iBin	rBin	nlog2 n	iMerge	rMerge
100	400	380	560	10000	4580	105960
1000	6000	420	620	1000000	22960	1083880
10000	90000	660	620	100000000	83100	7829940
100000	1100000	880	740	10000000000	671260	67335360
1000000	13000000	1280	980	1000000000000	6405460	705791140